

CS675

Computer Graphics

The Graphics Pipeline

Parag Chaudhuri



Department of Computer Science and Engineering
IIT Bombay



ViGIL
VISION, GRAPHICS,
IMAGING & LEARNING

What is an image to a computer?



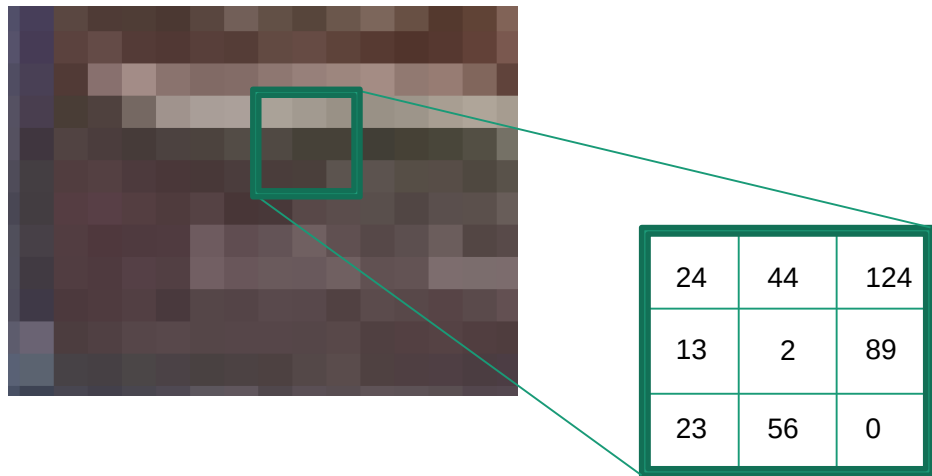
Image Source: <http://topyaps.com/kendriya-vidyalaya-facts>



Pixels



What is an image to a computer?



They are just arrays of numbers.

Images can be grayscale (1 number per array element) images or colour images (3 numbers per array element).

What are Computer Vision and Graphics?

- Learning how to fill the array with new numbers is **computer graphics**.
- Render, animate, synthesize



Toy story, Pixar

Is a computer good at creating images?

- Generative AI!
- Before there was DaLL-E and Stable Diffusion, there was rendering in computer graphics.

We will not talk about any AI/ML in this course. Only Computer Graphics.



Image formation in CG

Image Formation in CG

Rendering in CG tries to mimic how we take pictures in the real world.

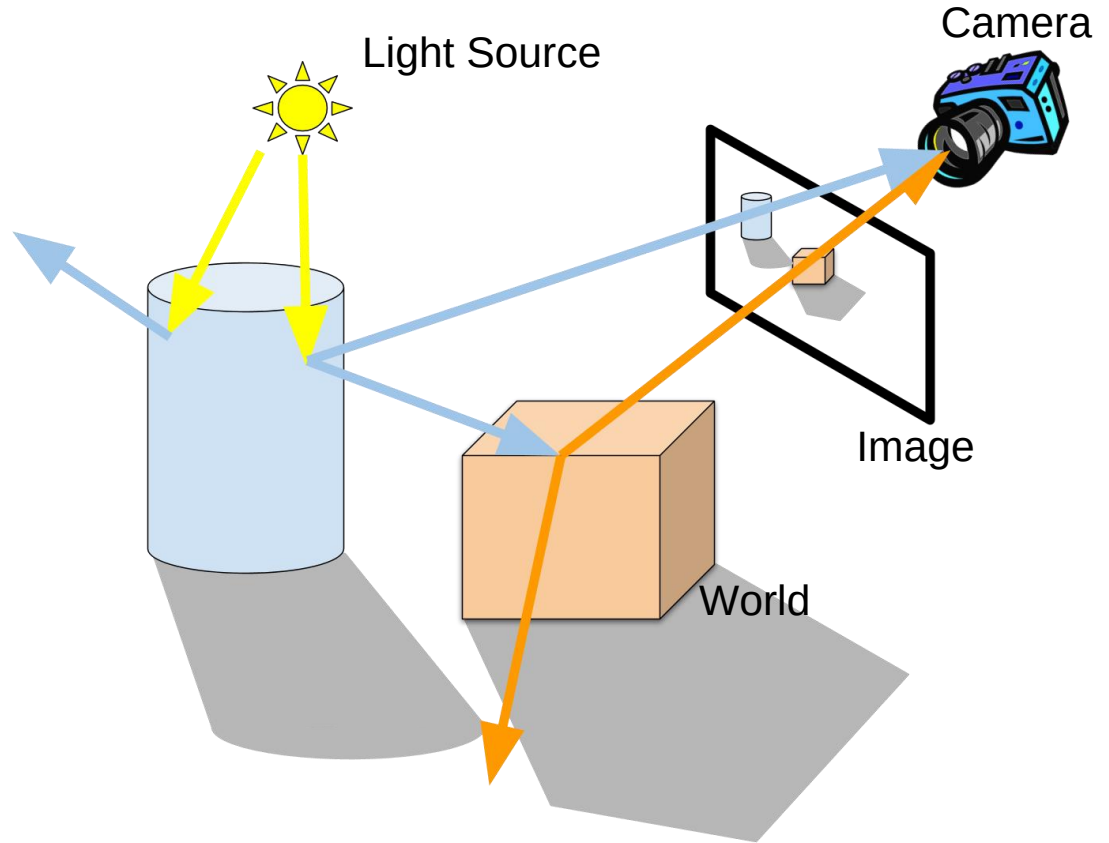


Image Formation in CG

Rays of light from a light source bounce from the world and reach a camera.

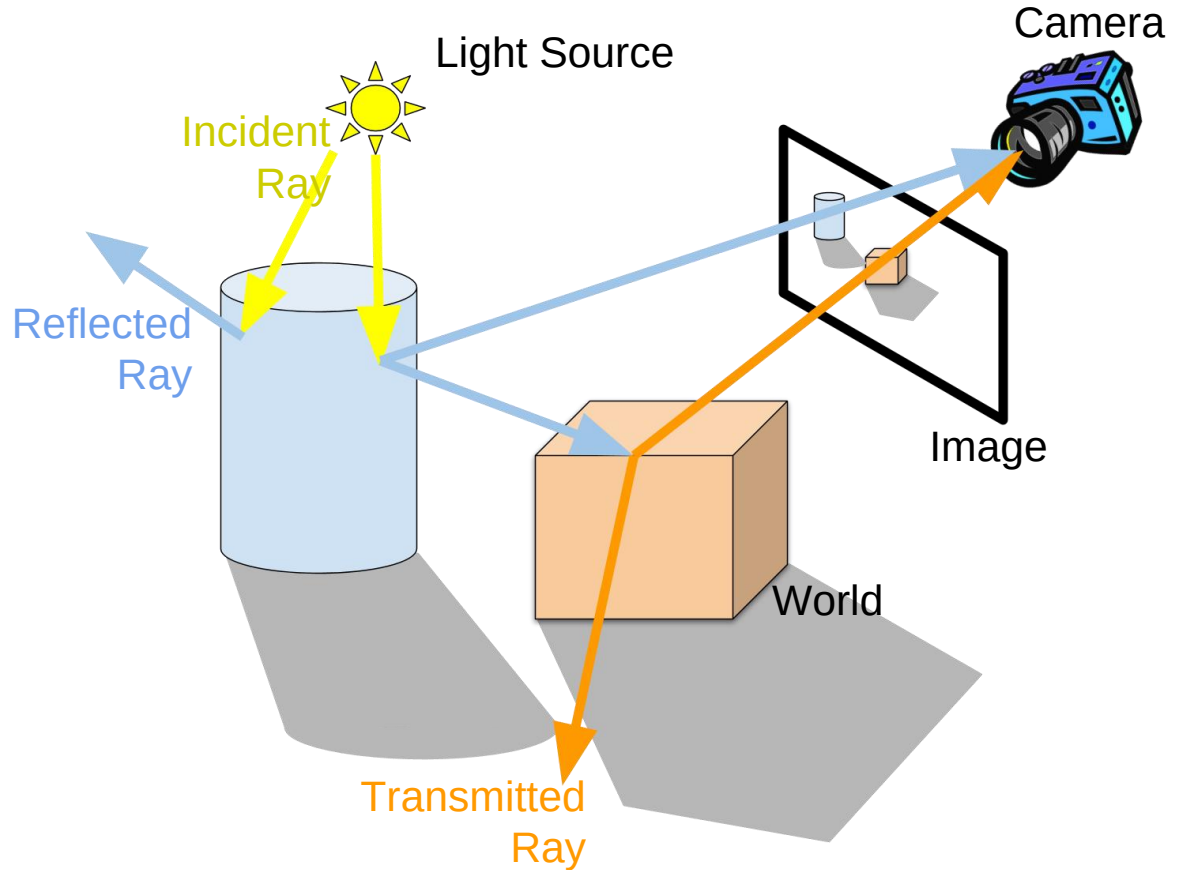


Image Formation in CG

Modelling
Assumptions

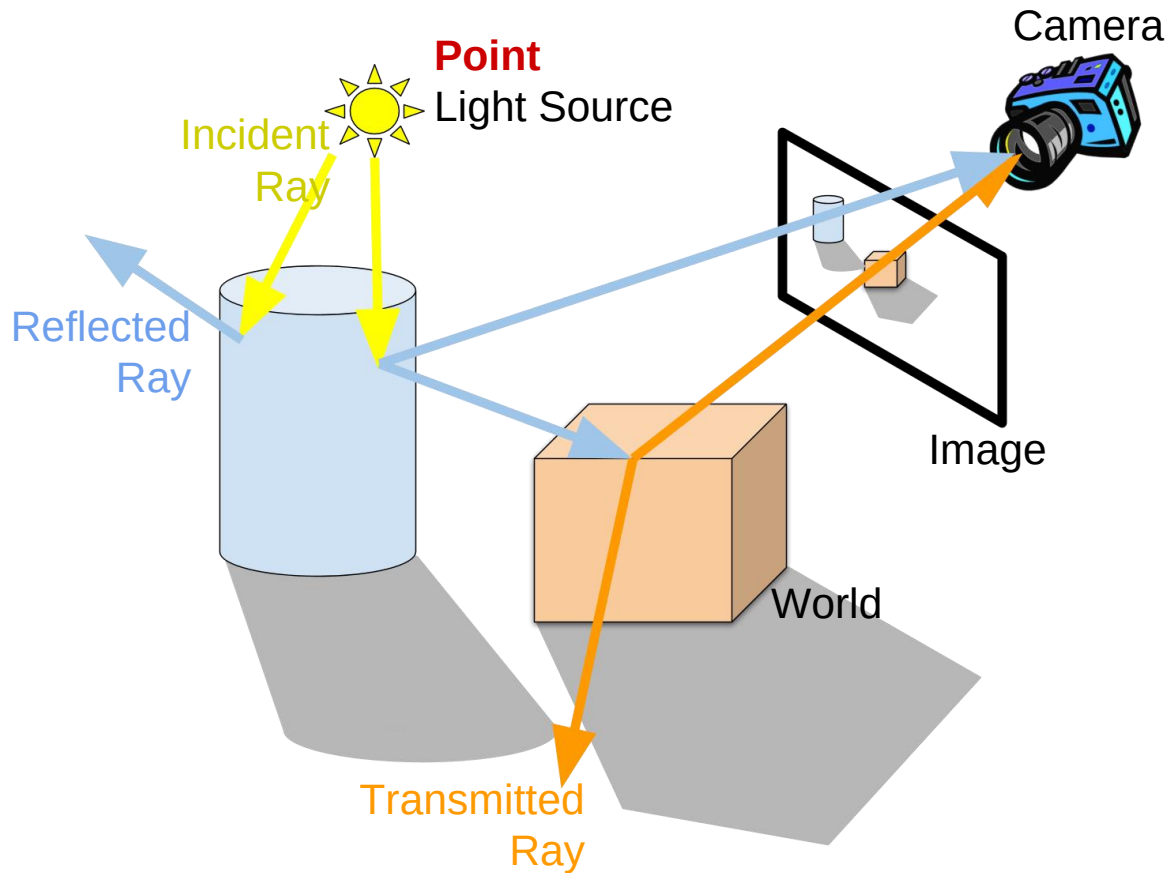


Image Formation in CG

Modelling
Assumptions

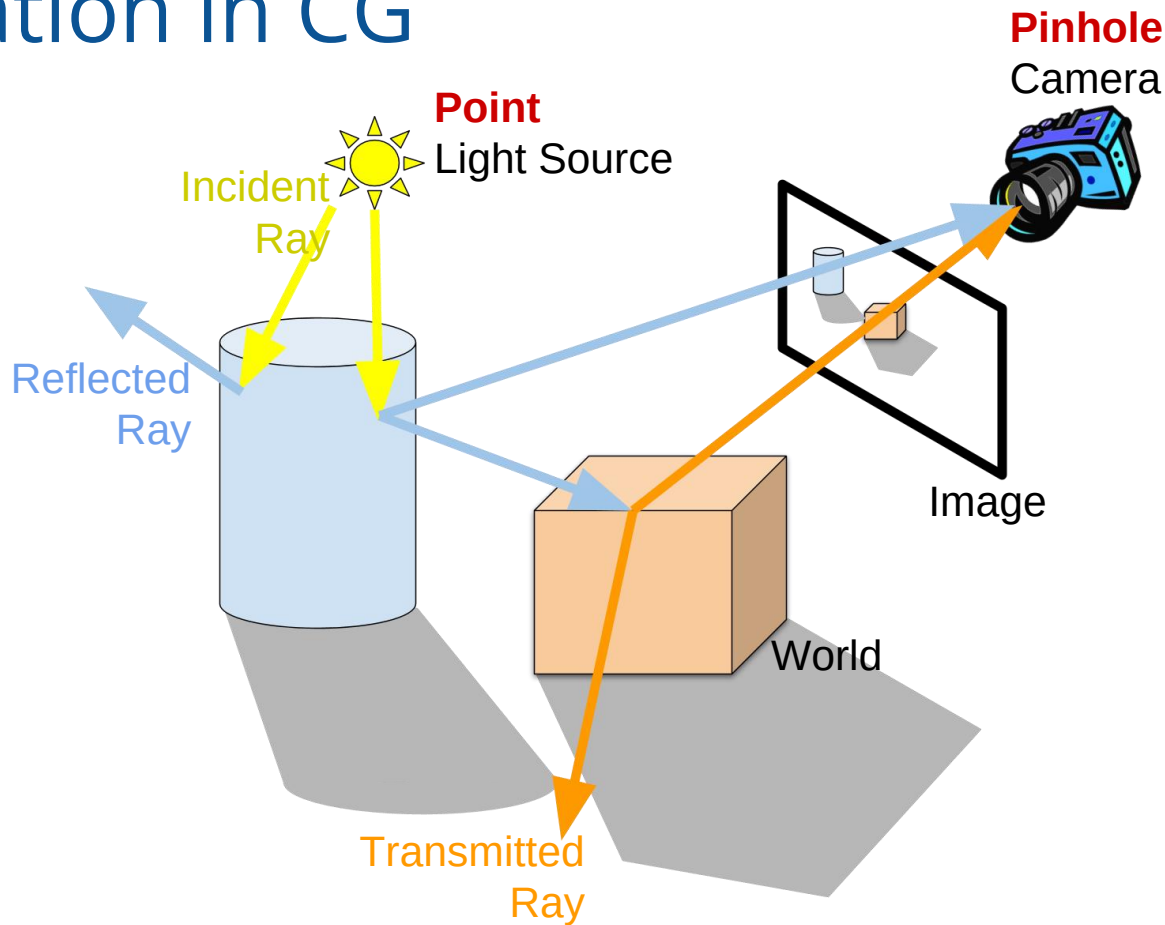


Image Formation in CG

Modelling Assumptions

For the objects in the world, we have their geometry and their material properties that consists of surface color and various material constants like refractive index and reflectance.

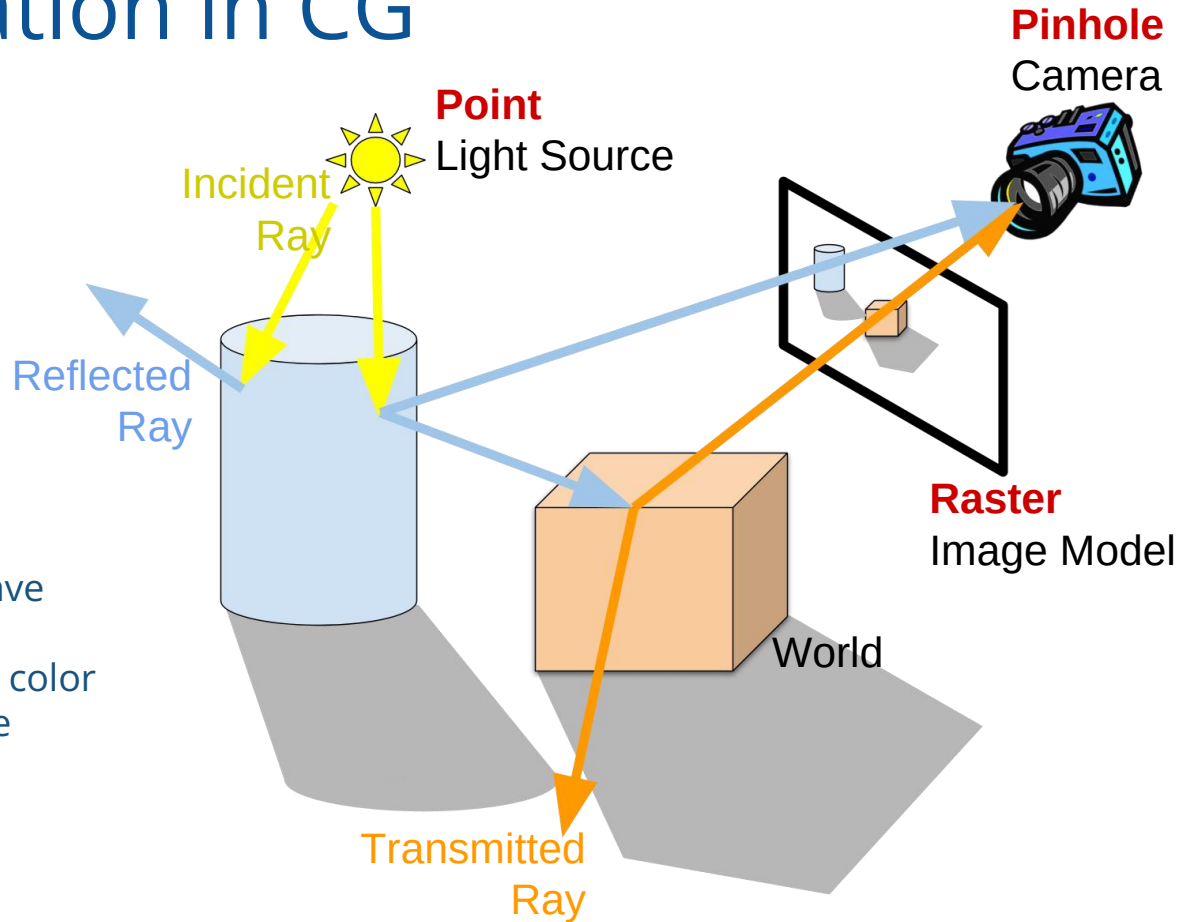


Image Formation in CG

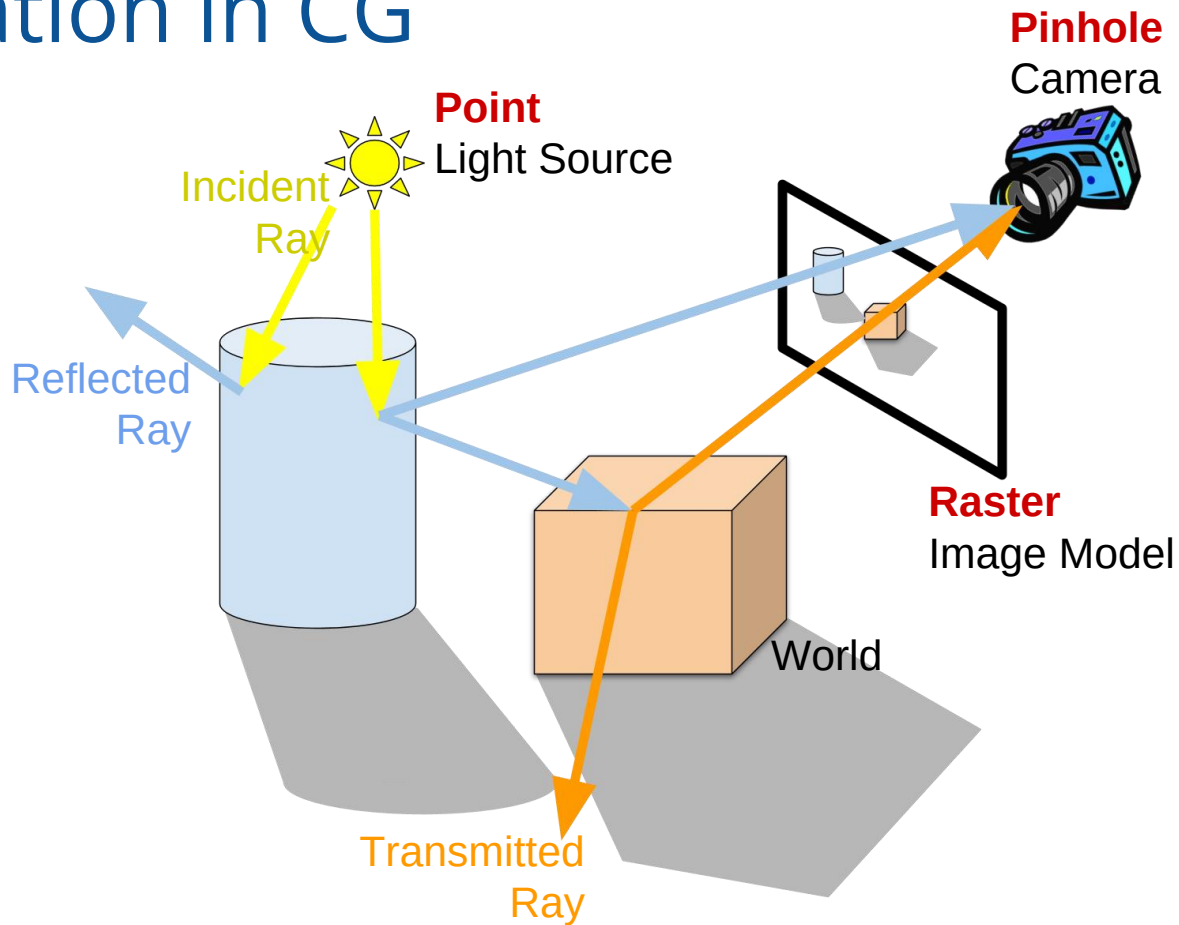


Image Formation in CG

How is this
image drawn
to the screen ?

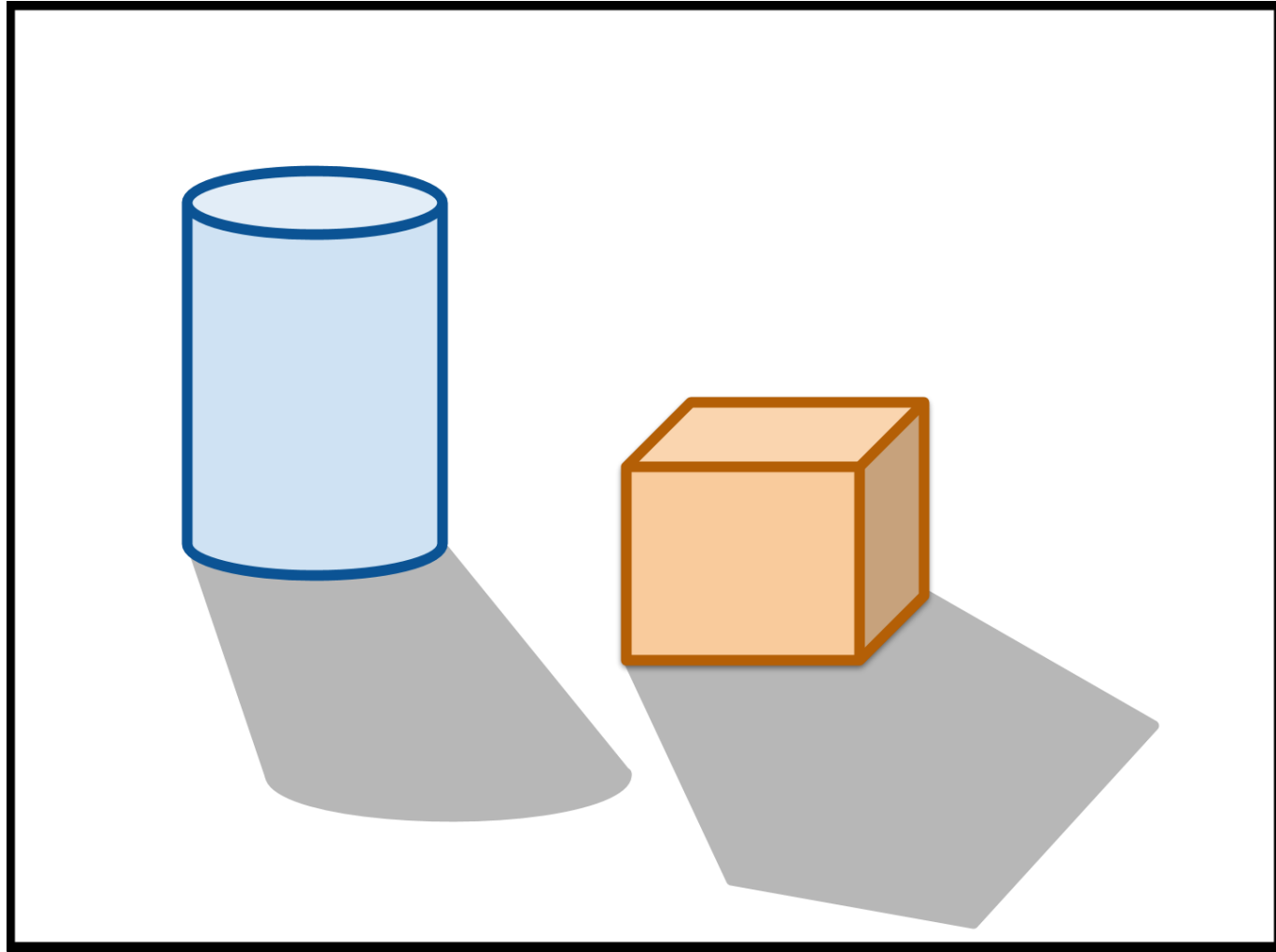
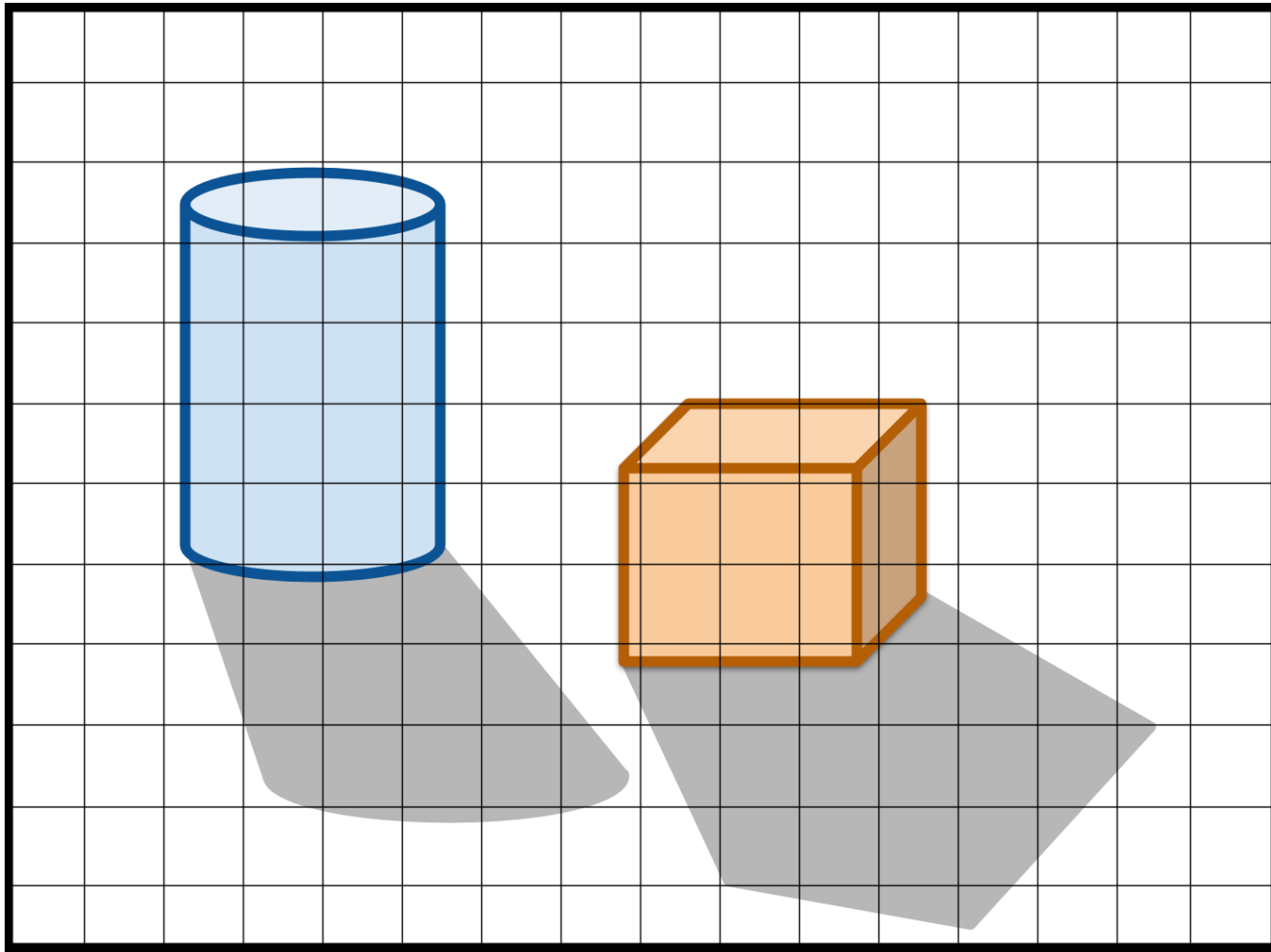


Image Formation in CG

The image is an array of *picture elements* or *pixels*.

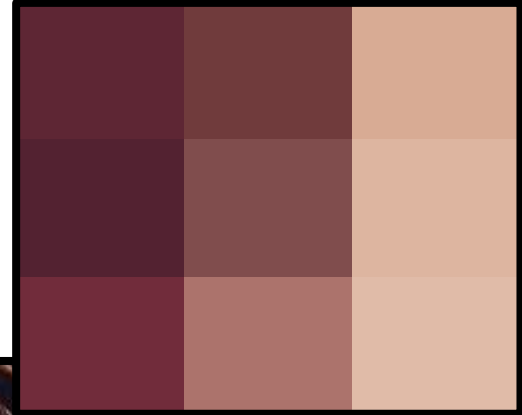
Each pixel has at least a colour value.

We need to figure out which pixels to colour and with what colour.



So what is Rendering in CG?

- Create a discrete view of a continuous world
- Colour the *pixels*



 blender

Two classes of algorithms for rendering

Rasterization

For each object

For each pixel

Determine if closer.

Colour Pixel.

Raytracing

For each pixel

For each object

Determine if closest.

Colour Pixel.

All rendering algorithms have to solve two primary problems –

1. Visibility (what is visible).
2. Illumination transfer and Shading (what color to give to the pixel).

Rasterization Graphics Pipeline

A hardware abstraction for real-time computer graphics

The Graphics Pipeline

- Real-time Computer Graphics – Games, VR, any interactive visual content.
- Pipelining as an idea to increase throughput/speed.
- A functional description

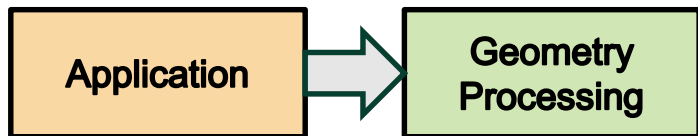
The Graphics Pipeline – Stage 1



Application

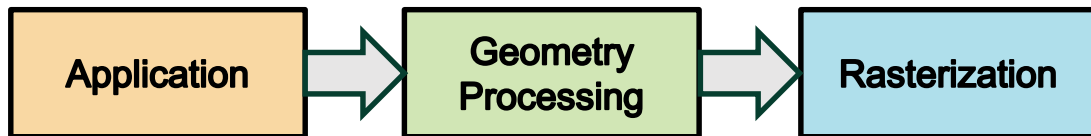
- The application stage executes mostly on the CPU
 - Global acceleration algorithms
 - Collision detection/response
 - Animation/Physics
 - Networking
 - Use input handling

The Graphics Pipeline – Stage 2



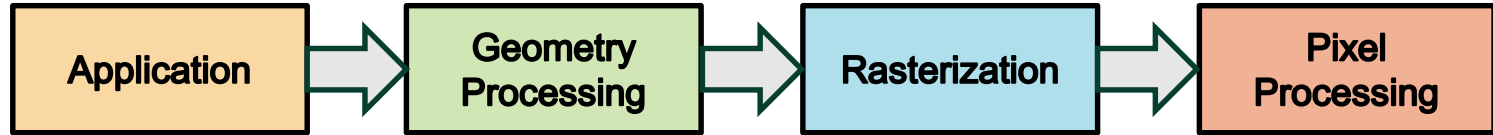
- The geometry processing stage executes on the GPU
 - Computes what geometry is to be drawn, where and how.
 - It deals with transforms, projections and vertex level computations.
 - These GPU stages are known as *Vertex Shaders or Processors*, and are completely programmable in modern GPUs.

The Graphics Pipeline – Stage 3



- The rasterization executes on the GPU
 - It assembles the primitives – points, lines and triangles – from incoming streams of vertices.
 - This part is completely hardcoded on the GPU hardware and cannot be programmed.
 - The rasterizer computes the pixels that the primitives cover on the screen, and emit them in a stream as output.

The Graphics Pipeline – Stage 4



- The pixel processing stage also runs on the GPU
 - The pixel shader or processor computes colour for every pixel.
 - It also determines pixel level visibility, to determine which pixel to draw and what to discard.

Why the raster image model makes sense?



Display



Framebuffer



Processor



Input

The framebuffer is a memory buffer that stores the colour value for every pixel that is displayed.

Why the raster image model makes sense?



Display



Framebuffer



Processor



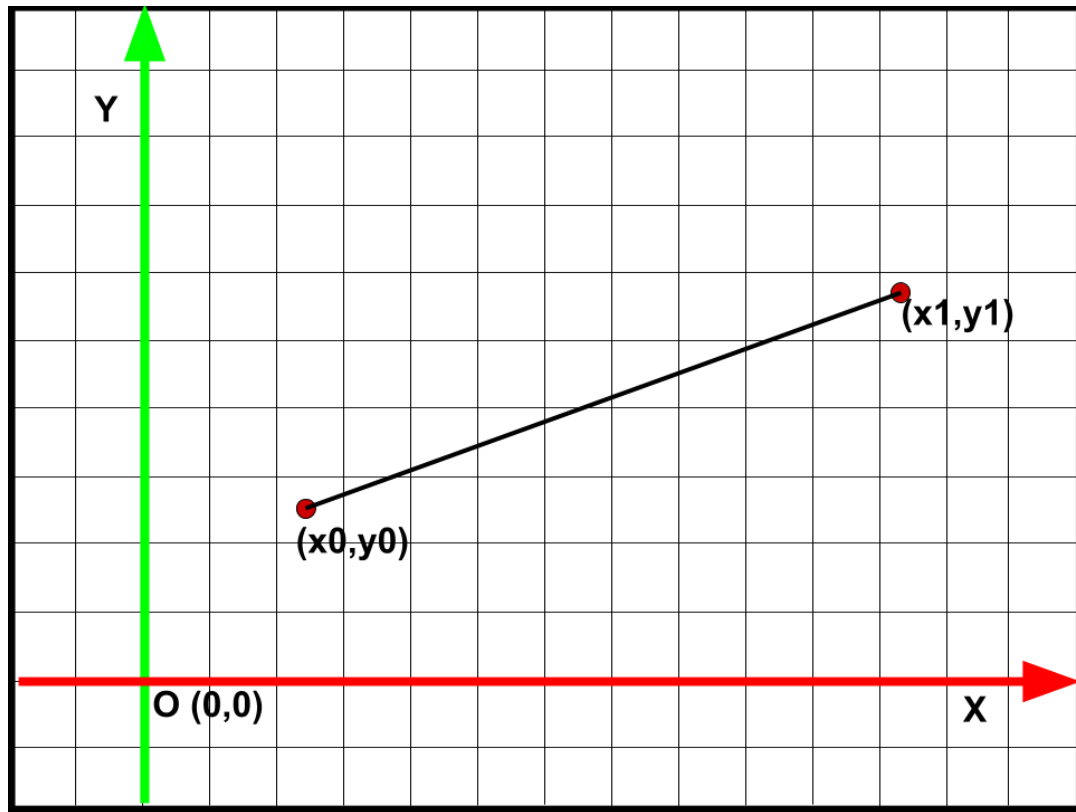
Input

So the image model mimics the hardware memory model.

The makes loading and manipulating these images in memory simpler.

Rasterizing Lines and Polygons

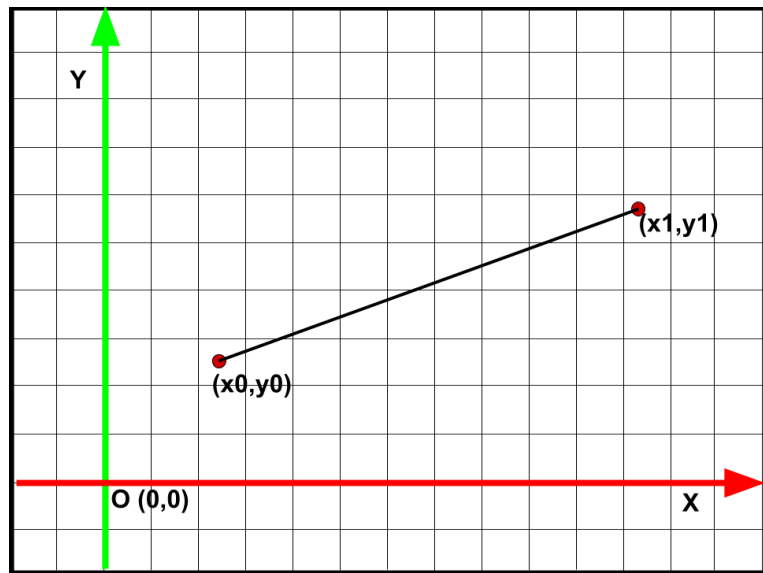
How to colour the correct pixels?



How to rasterize a line (segment)?

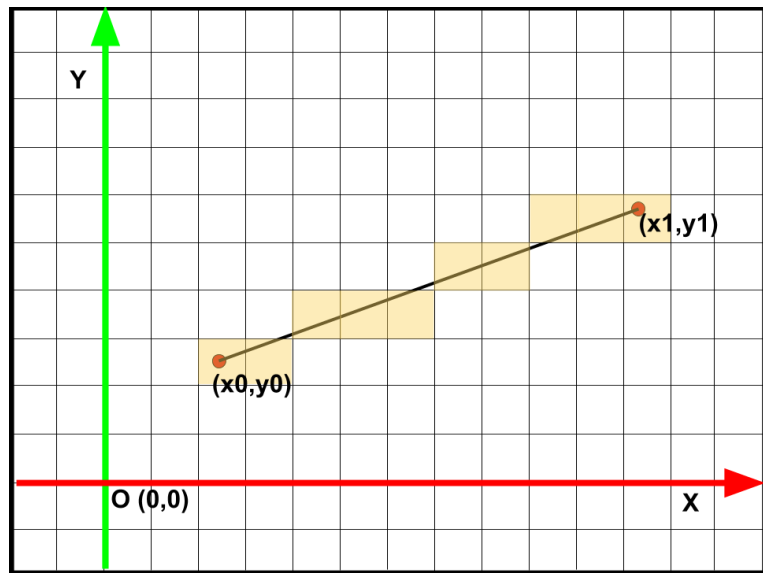
- Figure out which pixels does the geometric line pass through?
- A notion of correctness
- Efficient - Millions of lines may be drawn in a scene.
- No overdrawing of pixels.

Bresenham Line Drawing



```
function line(int x0, int x1, int y0, int y1)
    int deltax = x1 - x0
    int deltay = y1 - y0
    float error = 0
    float deltaerr = deltay / deltax
        // Assume deltax != 0 (line is not vertical),
        // note that this division needs to be done in a way
        // that preserves the fractional part
    int y = y0
    for x = x0 to x1
        plot(x,y)
        error = error + deltaerr
        if error >= 0.5 then
            y = y + 1
            error = error - 1.0
```

Bresenham Line Drawing



```
function line(int x0, int x1, int y0, int y1)
    int deltax = x1 - x0
    int deltay = y1 - y0
    float error = 0
    float deltaerr = deltay / deltax
    // Assume deltax != 0 (line is not vertical),
    // note that this division needs to be done in a way
    // that preserves the fractional part
    int y = y0
    for x = x0 to x1
        plot(x,y)
        error = error + deltaerr
        if error >= 0.5 then
            y = y + 1
            error = error - 1.0
```

Bresenham Line Drawing

- Extend to all quadrants, and replace floating point computations with integers.
- If we consider the distance between the actual geometric line and the pixel centers chosen by the algorithm, show that the Bresenham solution minimizes the *sum of squares of these distances*.
- The objective function of this minimization can be considered to be representative of the total error that the algorithm incurs when rasterizing a line.
- Check out the Bresenham demo.

Rasterizing Curves

Circle

This is an implementation of the circle algorithm.

```
void plotCircle(int xm, int ym, int r)
{
    int x = -r, y = 0, err = 2-2*r; /* II. Quadrant */
    do {
        setPixel(xm-x, ym+y); /* I. Quadrant */
        setPixel(xm-y, ym-x); /* II. Quadrant */
        setPixel(xm+x, ym-y); /* III. Quadrant */
        setPixel(xm+y, ym+x); /* IV. Quadrant */
        r = err;
        if (r <= y) err += ++y*2+1; /* e_xy+e_y < 0 */
        if (r > x || err > y) err += ++x*2+1; /* e_xy+e_x > 0 or no 2nd y-step */
    } while (x < 0);
}
```

Source: <http://members.chello.at/~easyfilter/bresenham.html>

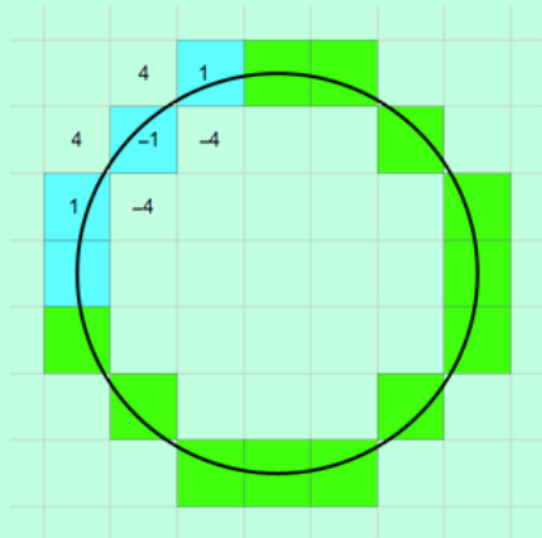
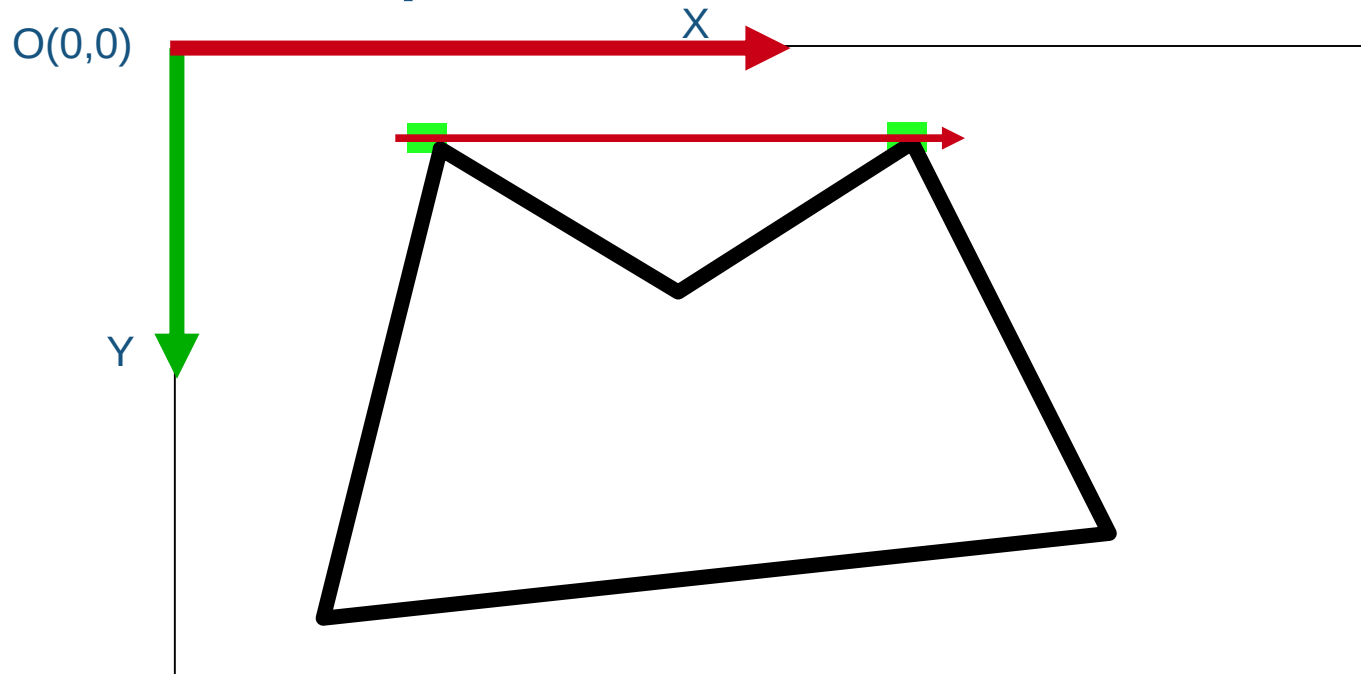
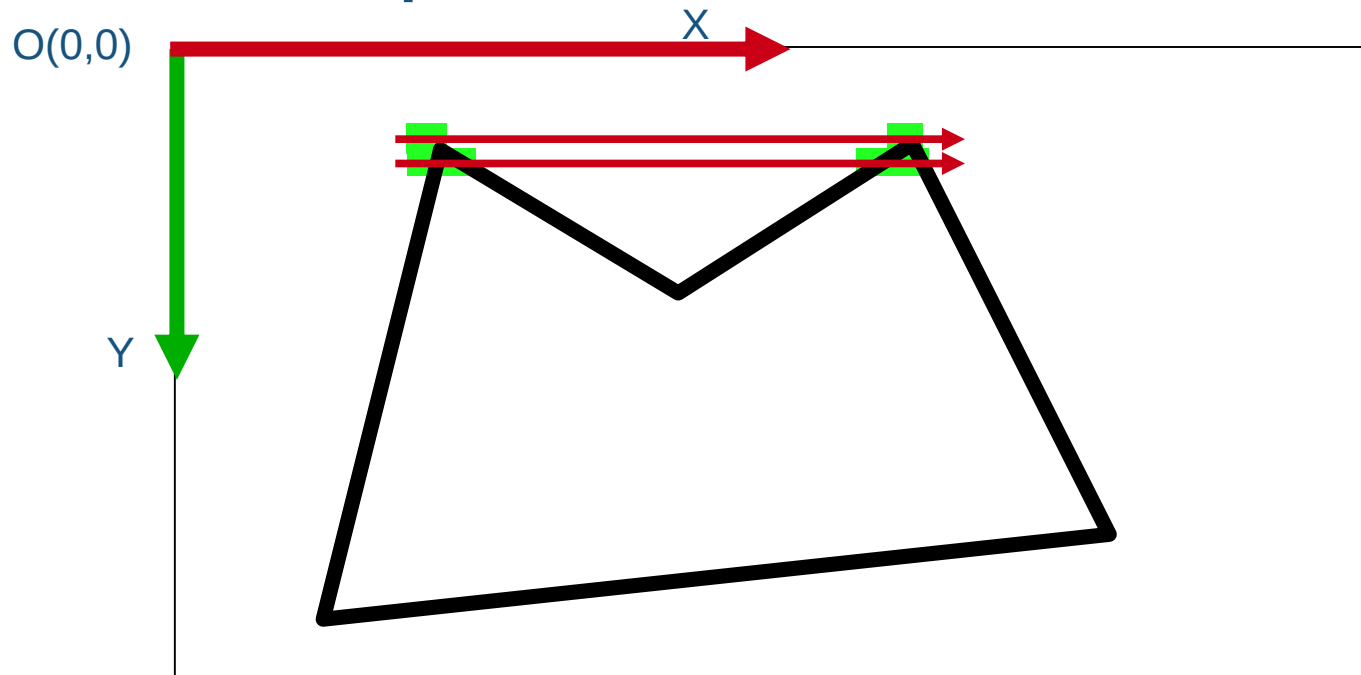


Figure out why and how does this work ?

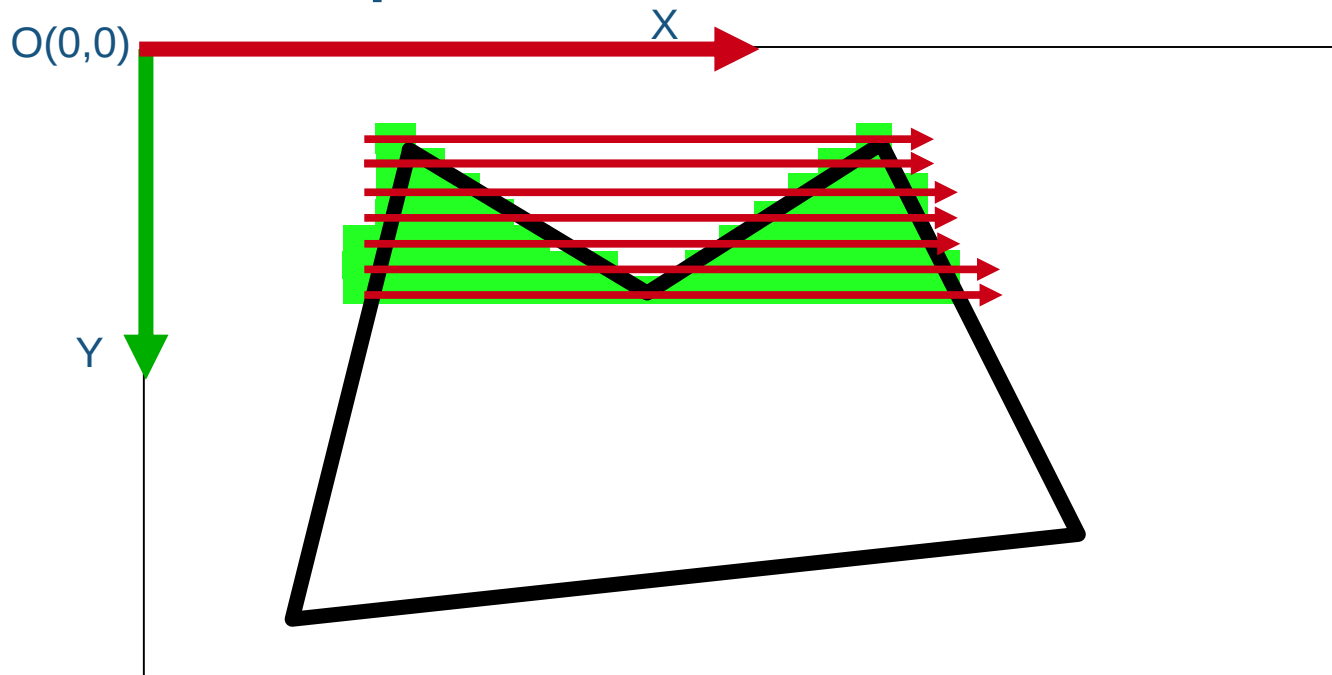
How to fill pixels?



How to fill pixels?



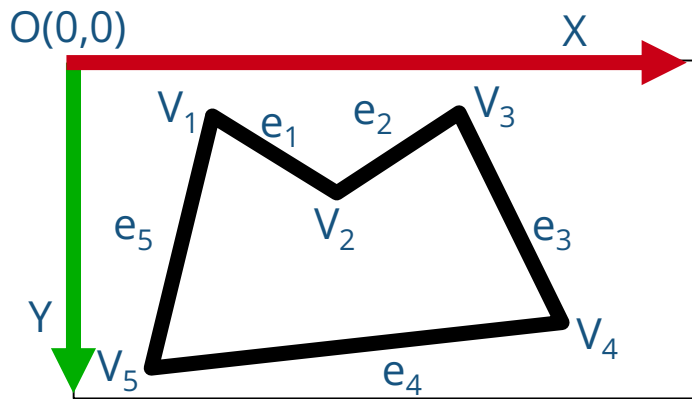
How to fill pixels?



Scanfill Algorithm

Scanfill Algorithm

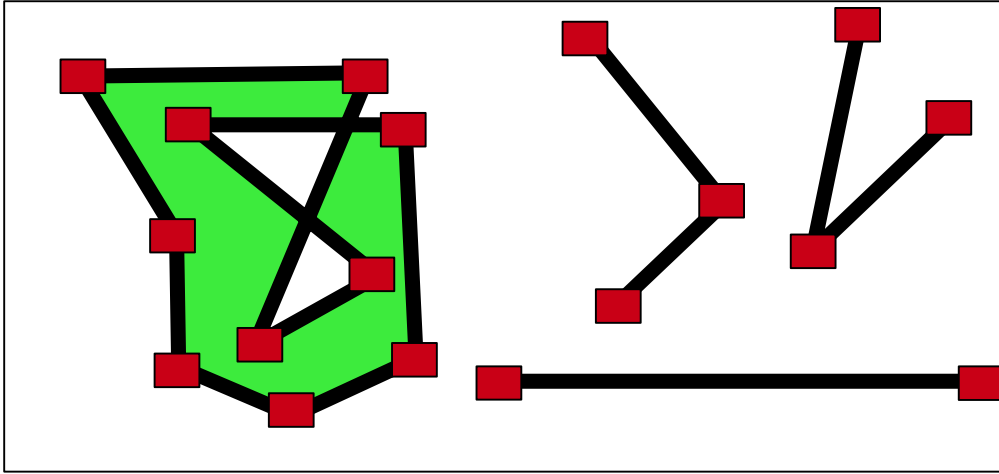
The Edge List



Edge	$Y_{\min}$	$Y_{\max}$	X for $Y=Y_{\min}$	$1/m$
e_2	V_3	V_2	X_3	$1/m_2$
e_3	V_3	V_4	X_3	$1/m_3$
e_1	V_1	V_2	X_1	$1/m_1$
e_5	V_1	V_5	X_1	$1/m_5$
e_4	V_4	V_5	X_4	$1/m_4$

- Edges in the edge list become *active* when the y-coordinate of the current scan line matches their $Y_{\min}$ value.
- First intersection point between an active edge and a scan line is always the endpoint corresponding to $Y_{\min}$.

Scanfill Algorithm

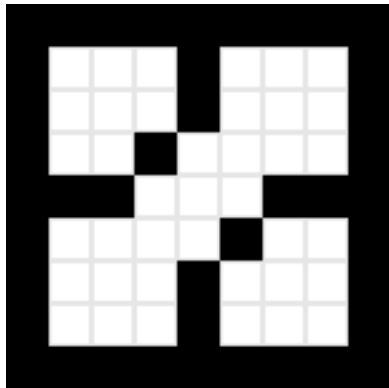


- Inside / Outside
- Special Cases

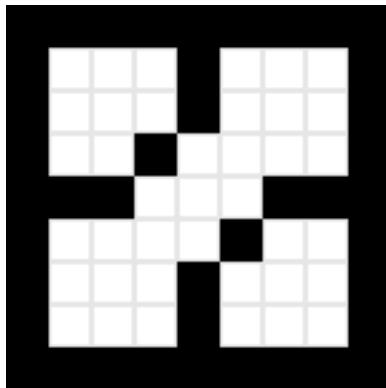
- For monotonically increasing/decreasing edges across a shared vertex count *one* intersection.
- Else count *two*.
- Ignore horizontal edges.

FloodFill

- A seed pixel is clicked.
- Neighbouring pixels are recursively filled till
 - everything in the neighbourhood is coloured or is a boundary



4 pixel
neighbourhood

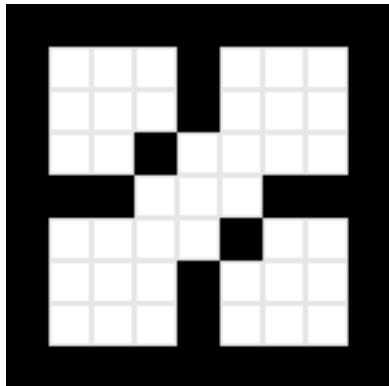


8 pixel
neighbourhood

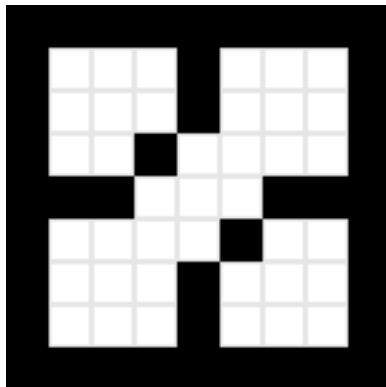
Source: https://en.wikipedia.org/wiki/Flood_fill

FloodFill

- A seed pixel is clicked.
- Neighbouring pixels are recursively filled till
 - everything in the neighbourhood is coloured or is a boundary



4 pixel
neighbourhood



8 pixel
neighbourhood

Source: https://en.wikipedia.org/wiki/Flood_fill

Is recursion a good way
to implement this ?

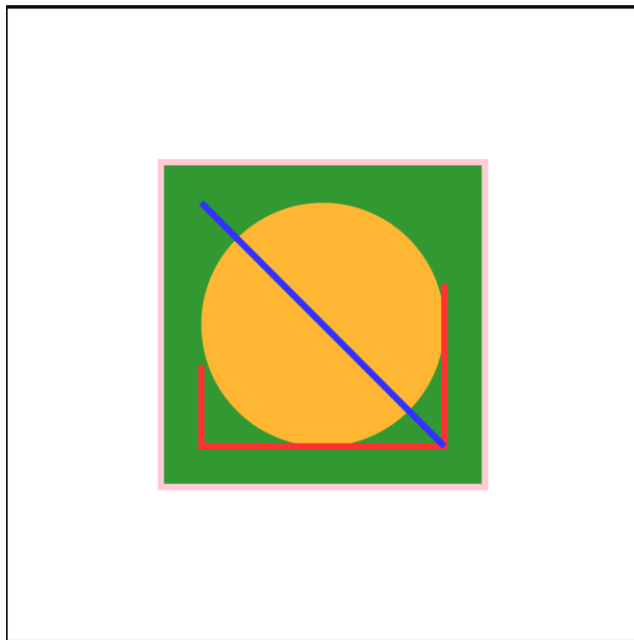
Alternative Image Models

Vector Images

Vector Images

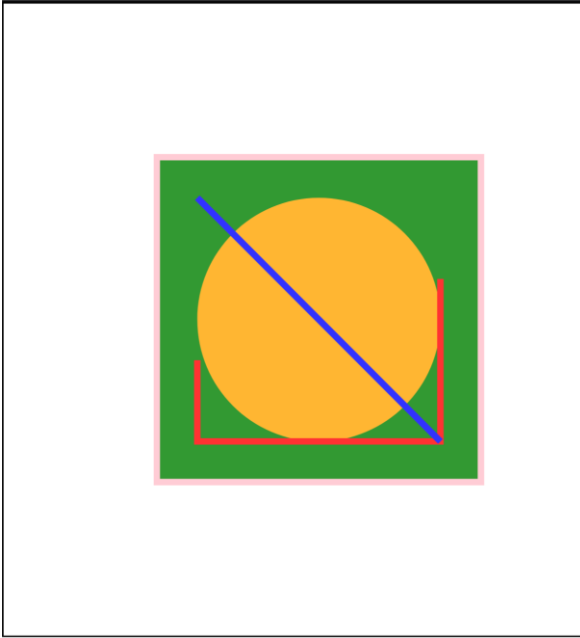
- An alternative to raster images, are vector images.
- Vector images store points coordinates, which are joined by lines or curves to form shapes.

Vector Images



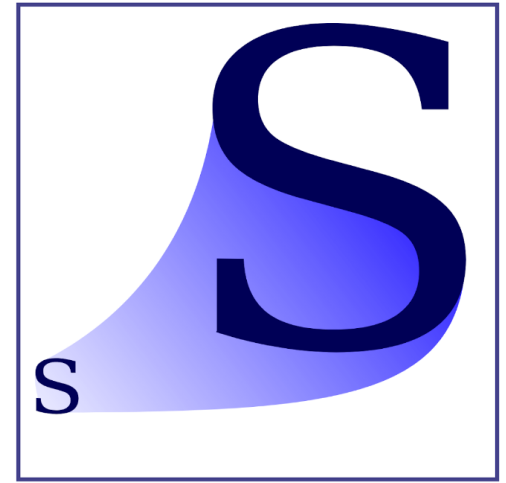
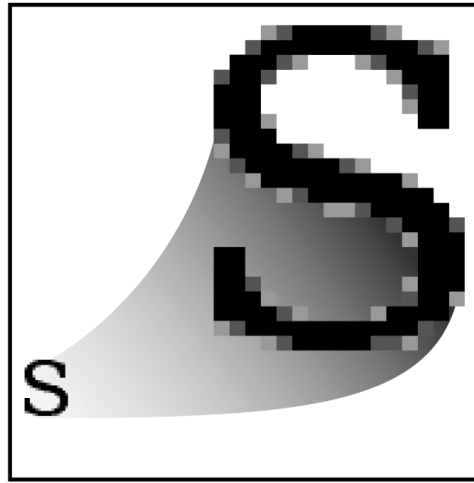
https://en.wikipedia.org/wiki/Scalable_Vector_Graphics

```
1 <?xml version="1.0" encoding="UTF-8" standalone="no"?>
2 <svg width="391" height="391" viewBox="-70.5 -70.5 391 391" xmlns="http://www.w3.org/2000/svg">
3 <rect fill="#fff" stroke="#000" x="-70" y="-70" width="390" height="390"/>
4 <g opacity="0.8">
5   <rect x="25" y="25" width="200" height="200" fill="green" stroke-width="4" stroke="pink" />
6   <circle cx="125" cy="125" r="75" fill="orange" />
7   <polyline points="50,150 50,200 200,200 200,100" stroke="red" stroke-width="4" fill="none" />
8   <line x1="50" y1="50" x2="200" y2="200" stroke="blue" stroke-width="4" />
9 </g>
10 </svg>
```


[illegible]

Here are first 14 rows of pixels when the same image is rasterized.

Vector Images



https://en.wikipedia.org/wiki/Scalable_Vector_Graphics

- Vector images are resolution independent. Perfect for icons, logos and fonts.
- Vector images must be rasterized for display.
- Is a file storing an image in a vector image format (e.g., SVG) smaller or larger than one stored in a raster image format like (e.g., JPG or PNG)?

Video Lectures

- Watch units 1-2, 1-3, 1-4 for introduction to graphics
- Watch units 2-1, 2-2, 2-3, 2-4 for image models and rasterization

Note: Video lectures do not cover everything discussed in class. So please treat the assigned viewing as mandatory supplementary material.